

05 — Client Core & UI

05 — Client Core & UI

Revision: 1 **Last modified:** 2026-07-05T14:20:00Z **Status:** Draft
— consolidated from 03-client-core-and-ui.md and v04-client/*.

1. Position

This section covers the stack from the **Rust helix-core FFI surface** up through the **Flutter helix-ui apps** and the **per-platform tunnel shims**.

The client stack consumes:

- Data-plane contracts from 01-data-plane.md (Transport trait, TunnelStatus, RouteMap).
- Control-plane contracts from 02-control-plane.md (REST/WS, WatchNetworkMap, enrollment).

It owns everything from the FFI surface upward.

2. Shared-codebase strategy

Layer	Shared by	Technology
Rust core (helix-core)	Access, Connector, Gateway edge	Rust — WG, transport, reconciler, kill-switch
Flutter UI (helix-ui)	Access, Connector, Console	Dart/Flutter — one tree, three flavors
Per-platform shim	Access/Connector per OS	Swift/Kotlin/C++/Qt — creates TUN, hands fd to core

Console is **API-client only** — it links no Rust tunnel core.

3. helix-core Rust workspace

```
helix-core/
├── crates/
│   ├── helix-transport/  # Transport trait + carriers
│   ├── helix-wg/         # boringtun wrapper
│   ├── helix-tun/        # TUN abstraction
│   ├── helix-route/      # RouteMap, CompiledPolicy
│   ├── helix-daifa/      # optional shaping
│   └── helix-core/       # orchestrator + reconciler + state
└── machines
    ├── helix-ffi/        # FFI surface (staticlib/cdylib)
    └── bin/
        └── helixd.rs     # --mode={client|connector} daemon
```

4. FFI surface (helix-ffi)

Dart (via flutter_rust_bridge v2) and native shims (via UniFFI / C-ABI) call:

```
pub fn helix_start(cfg: ClientConfig) -> Result<(), CoreError>;
pub fn helix_stop() -> Result<(), CoreError>;
pub fn helix_set_exit(selection: ExitSelection) -> Result<(),
    CoreError>;
pub fn helix_pin_transport(kind: Option<TransportKind>) ->
    Result<(), CoreError>;
pub fn helix_status_stream(sink: StreamSink<TunnelStatus>);
pub fn helix_set_shields(shields: ShieldConfig) -> Result<(),
    CoreError>;
pub fn helix_advertise(cidrs: Vec<IpNet>) -> Result<(),
    CoreError>;
pub fn helix_attach_tun(fd: RawFd) -> Result<(), CoreError>;
```

TunnelStatus is the hot status stream consumed by the UI:

```
pub enum TunnelStatus {
    Disconnected,
    Connecting { transport: Option<String> },
    Handshaking,
    Connected { transport: String, rtt_ms: u32 },
    Reconnecting,
    Down { reason: String },
}
```

5. Size and memory build strategy

The iOS Network Extension memory ceiling is the hardest budget:

```
[profile.release]
opt-level      = "z"
```

```
lto           = "fat"
codegen-units = 1
panic        = "abort"
strip        = true
```

- iOS uses staticlib + build-std with panic-immediate-abort for the smallest footprint.
- panic=abort means no catch_unwind at the FFI boundary; failures are returned as typed CoreError.
- Runtime: bounded packet-buffer pool, no per-flow state growth.

6. Flutter UI (helix-ui)

One tree, three flavors via runHelixApp(flavor, home, capabilities):

Flavor	Capabilities
Access	core_ffi, tunnel shim, connect UI
Connector	core_ffi, advertise/route config, optional slim UI
Console	no core_ffi; admin CRUD + live topology

State management: Riverpod providers; UI is a pure function of the FFI status stream.

7. Per-platform tunnel shims

Platform	Shim	Mechanism
iOS / macOS	shim-apple	NEPacketTunnelProvider
Android	shim-android	VpnService + JNI
Windows	shim-windows	wireguard-nt / wintun + privileged service
Linux	shim-linux	kernel WG + systemd
HarmonyOS NEXT	shim-harmonyos	Network Kit VPN ability + NAPI
Aurora OS	shim-aurora	Qt6/QML + C++ tun

The shim creates the TUN and hands the fd to helix-core via attach_tun(fd).

8. Connector mode

The Connector runs the **same Rust core** with `--mode=connector`:

- Does **not** capture the device's default route.
- Advertises served CIDRs.
- NATs/forwards decapsulated packets into the LAN interface.

9. Kill-switch + DNS

- Core-owned state machine drives the OS firewall.
- On tunnel drop or transport escalation, plaintext egress is blocked.
- DNS is forced through the tunnel; plaintext :53 off-tunnel is blocked.

10. Cross-references

- Detailed FFI → [../../v04-client/ffi-surface.md](#)
- Flutter UI → [../../v04-client/helix-ui-flutter.md](#)
- Core Rust packaging → [../../v04-client/helix-core-rust.md](#)
- Connector mode → [../../v04-client/connector.md](#)
- Platform shims → [../../v04-client/shim-*.md](#)
- State management → [../../v04-client/state-management.md](#)
- Design system → [10 — Design System](#)

Sources: docs/research/mvp/final/03-client-core-and-ui.md, v04-client/.md.*